Endpoint Inactivity Monitoring

Caso de uso demostrable con la API de Arexdata y Postman

Informe para la reunión de seguimiento

Fecha del informe	8 de octubre de 2026
Entorno	api.dev.eu.arexdata.com (desarrollo)
Endpoint utilizado	POST /Agent/reporting/endpoints
Herramienta	Postman (script post-response)

1. Resumen ejecutivo

Este caso de uso permite **identificar los endpoints que llevan más de 30 días sin conectarse a Arexdata**. Se apoya en un dato que la API ya devuelve (`lastConnectionAt`) y en un script de Postman que calcula la inactividad sobre la respuesta, sin necesidad de modificar el backend.

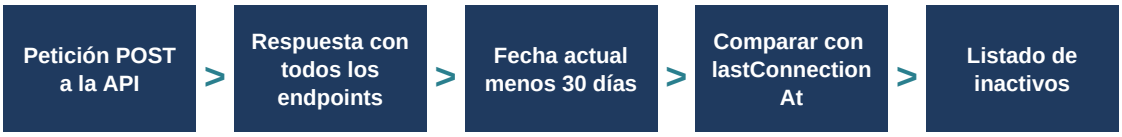
Pregunta	Respuesta
¿Qué problema resuelve?	Detectar agentes inactivos, equipos retirados o posibles problemas de conectividad.
¿Qué dato utiliza?	`lastConnectionAt` de cada endpoint.
¿Cuál es la regla?	Inactivo = sin conexión en los últimos 30 días (o sin ninguna conexión registrada).
¿Dónde se calcula?	En Postman, tras recibir la respuesta (post-response script).
¿Qué se obtiene?	Listado de endpoints inactivos, con máquina, última conexión y organización.

2. Justificación de negocio

Un agente que deja de reportar es un punto ciego: el equipo puede seguir en uso sin protección ni visibilidad, o haber sido retirado sin darse de baja. Disponer de un listado fiable de inactivos aporta:

- **Visibilidad**: los administradores saben qué equipos han dejado de comunicarse.
- **Higiene del inventario**: permite depurar equipos retirados o dados de baja.
- **Detección temprana de incidencias**: fallos del agente, problemas de red o desinstalaciones.
- **Cumplimiento y auditoría**: evidencia de que se revisan periódicamente los dispositivos gestionados.

3. Flujo de la solución



La API devuelve siempre todos los endpoints; el script de Postman hace el filtrado posterior. Por eso el cuerpo de la respuesta seguirá mostrando todos los equipos, algo normal y esperado.

4. Petición en Postman

Elemento	Valor
Método	POST
URL	`https://api.dev.eu.arexdata.com/Agent/reporting/endpoints`
Objetivo	Obtener todos los endpoints (el filtro del body se deja neutro).

Body (raw / JSON):

```
[
  {
    "filter": 0,
    "constraintsFilter": [0],
    "existsFilter": true,
    "numberFilter": 0,
    "dateFilter": 0,
    "column": "",
    "value": "",
    "noCaseInsensitive": true,
    "sort": 0
  }
]
```

5. Cómo funciona la lógica

Paso	Qué hace el script
1	Lee la respuesta de la API (lista de endpoints).
2	Toma la fecha y hora actuales.
3	Resta 30 días para obtener la **fecha límite de inactividad**.
4	Recorre cada endpoint y compara `lastConnectionAt` con la fecha límite.
5	Marca como inactivo si la última conexión es anterior o igual al límite, o si no existe el dato.
6	Muestra el resultado en la consola, lo guarda en la variable de entorno `inactiveMachines` y valida con un test de Postman.

6. Script de la pestaña Tests

Pegar el siguiente código en la pestaña **Tests** de la misma petición:

```
const machines = pm.response.json();
const now = new Date();

// 30 dias de antiguedad
const inactivityLimit = new Date(
    now.getTime() - (30 * 24 * 60 * 60 * 1000)
);

const inactiveMachines = machines.filter(machine => {
    if (!machine.lastConnectionAt) {
        return true;
    }
    const lastConnection = new Date(machine.lastConnectionAt);
    return lastConnection <= inactivityLimit;
});

console.log("ENDPOINT INACTIVITY MONITORING");
console.log("Current date:", now.toISOString());
console.log("Inactivity limit:", inactivityLimit.toISOString());
console.log("Total endpoints:", machines.length);
console.log("Inactive endpoints:", inactiveMachines.length);

console.table(
    inactiveMachines.map(machine => ({
        Machine: machine.machineName,
        LastConnection: machine.lastConnectionAt,
        Organization: machine.organizationId
    }))
);

pm.environment.set("inactiveMachines",
    JSON.stringify(inactiveMachines));

pm.test("Endpoint inactivity check completed", function () {
    pm.expect(inactiveMachines).to.be.an("array");
});
```

7. Cómo ejecutarlo y ver el resultado

- Pulsar **Send** en la petición.
- Abrir **View > Show Postman Console** para ver el informe de inactividad.
- Comprobar en la pestaña **Test Results** que el test aparece como superado.

Resultado esperado en el entorno actual

```
ENDPOINT INACTIVITY MONITORING
Current date: 2026-10-08T...
Inactivity limit: 2026-09-08T...
Total endpoints: 5
Inactive endpoints: 0
```

En el entorno de desarrollo la API devuelve **5 endpoints** y todos tienen `lastConnectionAt` del 8 de octubre, por lo que el resultado actual es **0 inactivos**. Esto confirma que la regla no genera falsos positivos.

8. Demostración de que la detección funciona

Como hoy no hay equipos inactivos, la lógica se demuestra con una prueba controlada: se **simula una fecha futura** (por ejemplo, 31 días después) para comprobar que los equipos pasan a considerarse inactivos.

Cambio puntual en el script (solo para la demo):

```
// Sustituir:
const now = new Date();

// Por (simula que han pasado 31 dias):
const now = new Date(Date.now() + (31 * 24 * 60 * 60 * 1000));
```

Con esa simulación, la fecha límite queda aproximadamente en la fecha actual real y todos los equipos, cuya última conexión fue hoy, quedan por debajo del límite: el script debe listar los **5 endpoints** como inactivos. Una alternativa equivalente es crear una segunda petición con datos simulados, donde un equipo tenga un `lastConnectionAt` antiguo.

Importante: tras la demo hay que devolver el script a la versión original (`new Date()`).

9. Alcance y limitaciones

Aspecto	Situación actual
Dónde se filtra	En Postman, **no en el backend**. La API sigue devolviendo todos los endpoints.
Automatización	Manual: se ejecuta al pulsar Send. Se puede programar con Postman Monitors o Collection Runner.
Umbral	30 días, fijo en el script. Es fácil parametrizarlo con una variable de entorno.
Equipos sin dato	Si `lastConnectionAt` no existe, se consideran inactivos.
Volumen	Al traer todos los endpoints, con inventarios muy grandes el filtrado local puede ser menos eficiente.
Zona horaria	Las fechas se comparan en formato ISO; conviene confirmar que la API las devuelve en UTC.

10. Siguientes pasos propuestos

- **Validar el caso de uso** con el equipo y confirmar que el umbral de 30 días es el adecuado.
- **Parametrizar el umbral** (por ejemplo, variable `inactivityDays`) para probar 7, 15 o 60 días.
- **Automatizar** la ejecución periódica y generar una alerta o informe cuando haya inactivos.
- **Evaluar la versión backend**: investigar si `/Agent/reporting/endpoints` puede filtrar directamente por fecha de última conexión mediante los campos `dateFilter`, `column` y `value` del body, o si hace falta un cambio en la API.
- **Documentar el caso de uso** en la colección de Postman para su reutilización en QA.

11. Conclusión

El caso de uso **Endpoint Inactivity Monitoring** es funcional, demostrable y fácil de justificar: con una sola petición existente y un script ligero se identifican los endpoints que no se conectan desde hace más de 30 días. Es una base válida para la demo y el QA, y el punto de partida para decidir si conviene convertirlo en una funcionalidad nativa de la API.

Resumen del caso de uso	
Endpoint	POST /Agent/reporting/endpoints
Dato utilizado	lastConnectionAt
Regla	Más de 30 días sin conexión
Procesamiento	Post-response (Postman)
Resultado	Listado de endpoints inactivos
Demo	Simulación de fecha futura